

SQL Practice Assignment - Northwind Database

Overview

This assignment provides a set of **SQL query challenges** based on the **Northwind** database.

Each student will automatically receive a **set of questions** generated based on their **student number**.

Your task is to **write SQL queries** that correctly answer each question, execute them using Python, and **submit your work by uploading a zip file containing the `Student_Number.txt` and the resulting JSON file (`query_hashes.json`)**.

Project Structure

```
project/
├──
├── databases/
│   ├── northwind.db      ← Main database for SQL queries
│   └── questions.db      ← Contains questions
├──
├── config/
│   └── Student_Number.txt ← Your student number (used to generate questions)
├──
├── questions/
│   ├── Question_0.py     ← Auto-generated question files (you will fill in answers here)
│   ├── Question_1.py
│   └── ...
├──
├── answer/
│   └── query_hashes.json  ← Auto-saved results of your answers (do not delete)
├──
└── generate_questions.py  ← The main generator script
```

Step 1 - Setup

1. Make sure you have **Python 3.10+** installed.
2. Install **Rich** for colored console output:

```
pip install rich
```

3. Verify that the following folders exist:

```
./databases
./config
./questions
./answer
```

4. Inside `./config`, make sure the file `Student_Number.txt` contains *only your student number*, for example:

```
1401012345678
```

Step 2 - Generate Your Questions

Run the `generate_questions.py`:

This will:

- Read your student number.
- select 12 questions for you.
- Create files named `Question_0.py`, `Question_1.py`, ..., in the `./questions` folder.

Each file will already include:

- The question text.

- The code to connect to the Northwind database.
- Code to save your answer hash automatically.

Step 3 - Answer Each Question

1. Open each file (e.g., `questions/Question_0.py`) in your editor.
2. Find the section:

```
# --- Fill in your SQL query here ---
answer = ''

...

# -----
```

3. Write your SQL query **inside** the triple quotes.
Example:

```
answer = '''
SELECT * FROM Employees;
'''
```

4. Save the file and run it
5. The output will:
 - Print your query results in a **Rich table**.
 - Automatically store a hash of your result in `answer/query_hashes.json`.

Step 4 - Verify Your Answers

You can check your progress in `answer/query_hashes.json`.
Each entry will look like this:

```
{
  "Q0": {
    "answer": "SELECT * FROM Employees;",
    "hash": "7e5b62f84a1e5..."
  },
  "Q1": {
    "answer": "...",
    "hash": "..."
  }
}
```

If `"hash"` is `"Failed"`, it means your query didn't run correctly (syntax or runtime error).

Step 5 - Submit

Submit only a zip file containing the following files/folders:

```
/config/Student_Number.txt
/answer/query_hashes.json
```

Do **not** include the database files or generator script.

Notes

- Do **not** modify anything outside the triple-quoted SQL area.
- Do **not** rename files or move them.
- Each question file must run successfully without errors.
- Your final `query_hashes.json` must contain 12 valid hashes.

Troubleshooting

Problem	Solution
<code>sqlite3.OperationalError: no such table</code>	Ensure <code>.db</code> files exist in <code>./databases</code>
<code>no such function: YEAR</code>	SQLite doesn't support <code>YEAR()</code> . Use <code>strftime('%Y', OrderDate)</code> instead.
adjust the <code>*_PATH</code> variables at the beginning of each file to fix file path related errors	